

Experiment 27: Best First Search Algorithm Using Prolog

Aim

To implement the **Best First Search algorithm using Prolog** and find a path from a starting node to a goal node using heuristic values.

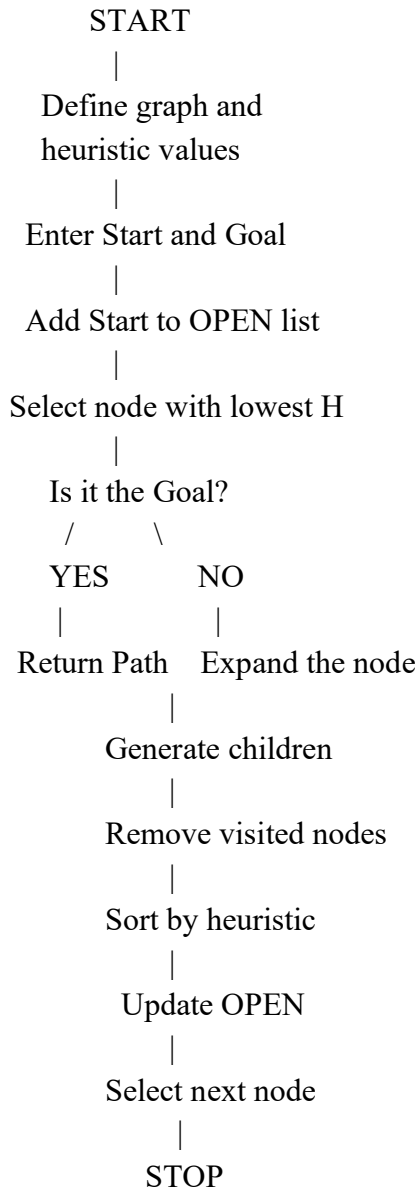
Objective

1. To understand the Best First Search algorithm.
2. To represent a graph using Prolog facts.
3. To assign heuristic values to different nodes.
4. To select the node with the lowest heuristic value.
5. To find a path from the starting node to the goal node.

Algorithm

1. Start the program.
2. Define the graph using edge facts.
3. Assign a heuristic value to each node using h facts.
4. Set the starting node and goal node.
5. Add the starting node to the open list.
6. Select the node with the lowest heuristic value.
7. Check whether the selected node is the goal.
8. If it is the goal, return the path.
9. Otherwise, expand the selected node and generate its neighboring nodes.
10. Add the unvisited neighboring nodes to the search list.
11. Sort the nodes according to their heuristic values.
12. Continue until the goal is reached.
13. Display the path.
14. Stop.

Flowchart



Prolog Code

```
% Best First Search
```

```
% Graph  
edge(a, b).  
edge(a, c).  
edge(b, d).
```

```
edge(b, e).  
edge(c, f).  
edge(c, g).  
edge(e, h).  
edge(f, h).
```

```
% Heuristic values
```

```
h(a, 10).  
h(b, 6).  
h(c, 4).  
h(d, 8).  
h(e, 3).  
h(f, 2).  
h(g, 5).  
h(h, 0).
```

```
% Best First Search
```

```
best_first(Start, Goal, Path) :-  
    search([Start], Goal, [], RevPath),  
    reverse(RevPath, Path).
```

```
search([Goal|_], Goal, Visited, [Goal|Visited]).
```

```
search([Current|Rest], Goal, Visited, Path) :-  
    findall(H-Next,  
        (edge(Current, Next),  
         \+ member(Next, Visited),  
         h(Next, H)),  
        Children),  
    sort(Children, Sorted),  
    remove_values(Sorted, NextNodes),  
    append(NextNodes, Rest, NewOpen),  
    search(NewOpen, Goal, [Current|Visited], Path).
```

```
remove_values([], []).
```

```
remove_values([_ -Node|T], [Node|R]) :-  
    remove_values(T, R).
```

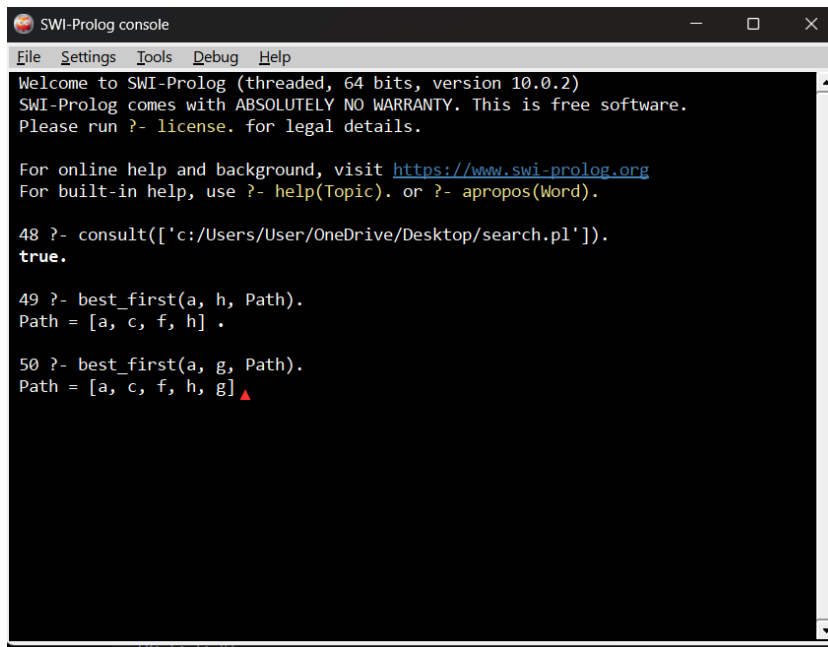
Sample Query

To find a path from a to h:

```
?- best_first(a, h, Path).
```

Another Query

```
?- best_first(a, g, Path).
```

A screenshot of the SWI-Prolog console window. The window has a title bar "SWI-Prolog console" and a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The console text is as follows:
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run `?- license.` for legal details.

For online help and background, visit <https://www.swi-prolog.org>
For built-in help, use `?- help(Topic).` or `?- apropos(Word).`

48 `?- consult(['c:/Users/User/OneDrive/Desktop/search.pl']).`
`true.`

49 `?- best_first(a, h, Path).`
`Path = [a, c, f, h] .`

50 `?- best_first(a, g, Path).`
`Path = [a, c, f, h, g] ▲`

Result

The **Best First Search algorithm** was successfully implemented using Prolog. The program used heuristic values to select nodes and successfully found a path from the starting node to the specified goal node.

Conclusion

Thus, the experiment demonstrates how **Best First Search can be implemented in Prolog using graph representation, heuristic values, recursion, and logical inference**. The algorithm selects the most promising node based on its heuristic value and continues searching until the goal is reached.